SECTION 3 – Business Insight Summaries

In this section, I wanted to go beyond basic data handling and focus on how meaningful insights can be derived from the gym's relational database. Each query was designed with a clear business purpose in mind – whether to track outstanding payments, understand class popularity, or assess trainer demand. Below, I've described the goal behind each query, what it reveals, and why it matters to the gym's operation

Members with Pending Payments

To support the gym's financial team, I created a query that retrieves a list of members who still owe payments. I joined the Members and Payments tables on MemberID and filtered for rows where the Status is marked as 'Pending'. This output includes each member's name, the date of the payment, the amount due, and the payment status.

This query is especially useful for identifying and following up with members who are behind on their fees.

Class Schedule by Trainer

This query presents the weekly class schedule, clearly organised by trainer. It joins the Trainers, ClassSchedule, and Classes tables, allowing me to pull together who is teaching what, and when. The result is ordered by day and time to reflect a real-world timetable.

This view can help with resource planning and ensures there are no clashes or overlaps in the schedule.

Most Booked Classes

To better understand member preferences, I ran a query that calculates which classes are booked the most. Using `GROUP BY`, I counted the number of bookings per class and sorted the results in descending order. I then limited the results to the top five.

This insight helps management identify the most popular classes, which could inform future scheduling and marketing strategies.

Revenue per Membership Type

Here, I examined the financial contribution of each membership type. I joined the Payments and Memberships tables, then used SUM and GROUP BY to calculate total revenue generated per membership category (e.g., Basic, Premium). *The gym can use this to evaluate which membership tier brings in the most income and consider whether pricing or benefits need to be adjusted.*

Attendance by Member

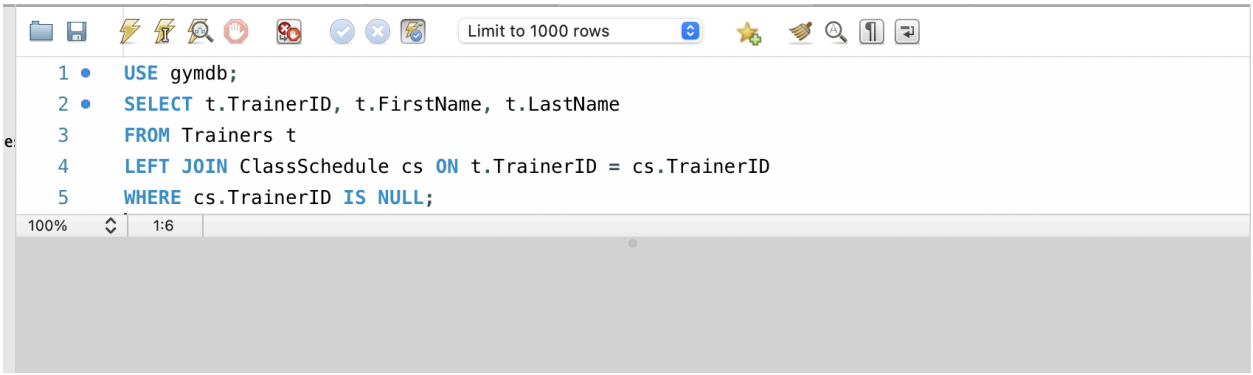
This query focuses on individual engagement, showing all the classes attended by a specific member. By filtering using MemberID and joining with the Attendance, Classes, and Trainers tables, I was able to generate a personal training history.

Useful for trainers who want to track a member's progress or for staff handling loyalty programmes.

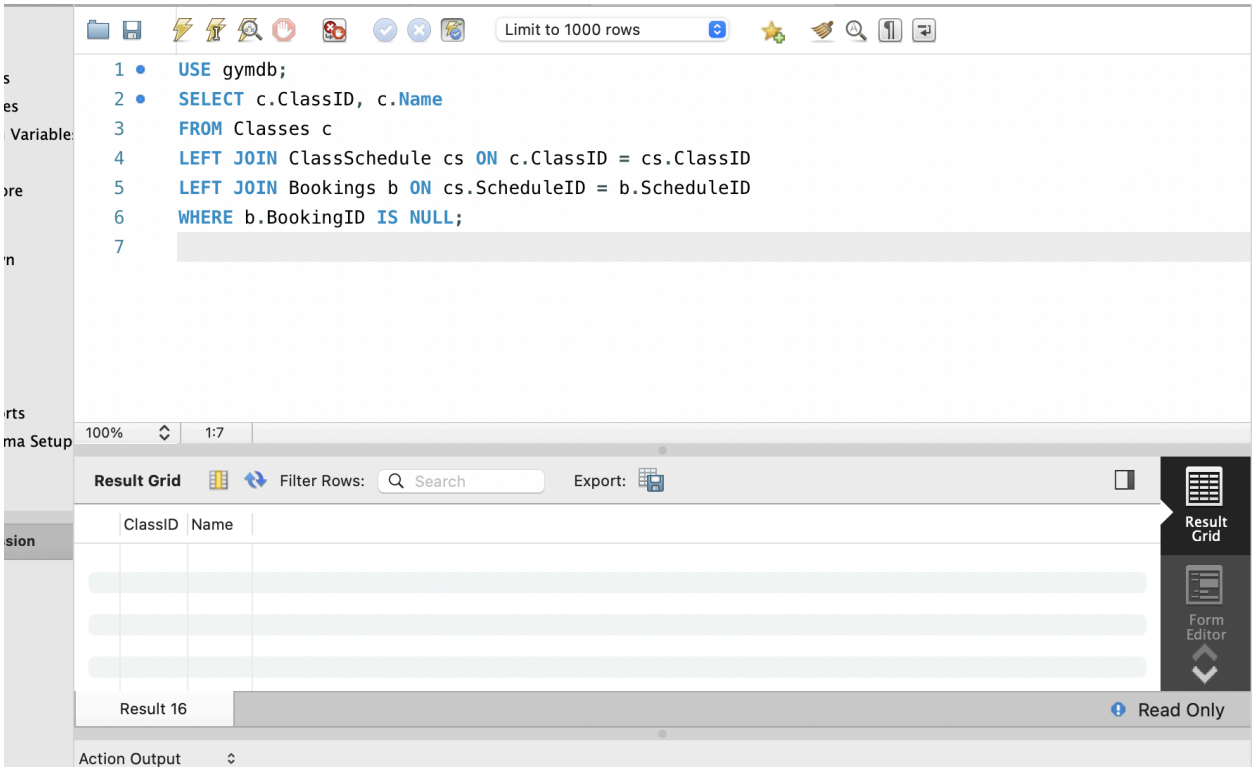
Most Booked Trainer

Lastly, I explored which trainer receives the most bookings. After grouping bookings by TrainerID and ordering them by count, I selected the top entry using LIMIT 1.

This is helpful in recognising high-performing staff and allocating resources fairly across the team.




Limit to 1000 rows
1 • USE gymdb;
2 • SELECT m.MemberID, m.FirstName, m.LastName, COUNT(b.BookingID) AS Bookings
3 FROM Members m
4 JOIN Bookings b ON m.MemberID = b.MemberID
5 GROUP BY m.MemberID, m.FirstName, m.LastName
6 HAVING COUNT(b.BookingID) > 1;
7
100%
1:7


Limit to 1000 rows
1 • USE gymdb;
2 • SELECT c.ClassID, c.Name
3 FROM Classes c
4 LEFT JOIN ClassSchedule cs ON c.ClassID = cs.ClassID
5 LEFT JOIN Bookings b ON cs.ScheduleID = b.ScheduleID
6 WHERE b.BookingID IS NULL;
7
100%
1:7
Result Grid
Filter Rows:
Search
Export:
ClassID
Name
Result Grid
Form Editor
Result 16
Read Only
Action Output

```sql
USE gymdb;
SELECT DATE_FORMAT(p.Date, '%Y-%m') AS PaymentMonth, SUM(p.Amount) AS TotalRevenue
FROM Payments p
WHERE p.Status = 'Paid'
GROUP BY PaymentMonth
ORDER BY PaymentMonth;
```

Limit to 1000 rows

100% 1:7

Result Grid Filter Rows: Search Export:

PaymentMonth	TotalRevenue
2025-05	60.00
2025-06	50.00

Result 17 Read Only

Action Output

Time Action Response Duration / Fetch Time

Query 1 gym_all_queries* gym_database* 01_create_member.png*

```sql
USE gymdb;
SELECT c.Name AS ClassName,
       COUNT(CASE WHEN b.Attendance = TRUE THEN 1 END) AS TotalAttended,
       COUNT(b.BookingID) AS TotalBooked,
       ROUND((COUNT(CASE WHEN b.Attendance = TRUE THEN 1 END) / COUNT(b.BookingID)) * 100, 2) AS A
FROM Bookings b
JOIN ClassSchedule cs ON b.ScheduleID = cs.ScheduleID
JOIN Classes c ON cs.ClassID = c.ClassID
GROUP BY c.Name;
```

Limit to 1000 rows

100% 17:9

Result Grid Filter Rows: Search Export:

ClassName	TotalAttend...	TotalBooked	AttendanceRate
Spinning	2	2	100.00
Yoga	1	2	50.00
HIIT	0	1	0.00

Result 19 Read Only

Action Output

Time Action Response Duration / Fetch Time